

Name: Deepak Kumar A
Reg. No.: 192524422
Date: 25-08-2026

1. Optimized Linux File-System Layout & Log Rotation

Suggested layout for 50 GB:

Partition	Size	Purpose
/	15 GB	OS and applications
/var	15 GB	Logs and variable data
/home	10 GB	User files
/tmp	5 GB	Temporary files
Swap	5 GB	Virtual memory

The /var partition is isolated so growing logs cannot fill the root filesystem. logrotate limits individual log growth.

```
Program: logrotate.conf
/var/log/myapp.log {
    size 1G
    rotate 9
    compress
    missingok
    notifempty
    copytruncate
}
```

Maximum retained log space: 1 GB × 10 = 10 GB.

README

Problem: Rapidly growing logs can consume the entire disk and stop critical services.

Solution: Separate /var from /, reserve 15 GB for it, and use logrotate to rotate, compress, and retain logs. With 9 rotated files plus the active 1 GB log, the configured log set stays within 10 GB.

Run:

```
sudo logrotate -f /etc/logrotate.d/myapp
```

2. Kernel-Level CPU Scheduling Optimization

The goal is to reduce unnecessary context switching and improve responsiveness for real-time workloads.

```
Program: scheduler_tune.sh
#!/bin/bash

# Reduce scheduler migration overhead
sudo sysctl -w kernel.sched_migration_cost_ns=500000

# Increase scheduler timeslice
sudo sysctl -w kernel.sched_latency_ns=24000000

# Give the real-time process priority
PID=$1
sudo chrt -f -p 80 "$PID"

echo "Scheduler tuning applied to PID $PID"
```

Run:

```
sudo ./scheduler_tune.sh <PID>
README
```

Problem: Excessive context switching reduces CPU efficiency in high-load systems.

Solution: Tune scheduler parameters to reduce unnecessary task migration and use chrt to assign real-time scheduling priority to critical processing tasks.

Note: Real-time priorities should be given only to carefully selected processes because excessive real-time priority can starve normal services.

3. Processing 1 Million Log Entries Within 2 Minutes

For large text files, avoid slow shell loops such as:

```
while read line
do
    ...
done
```

Use optimized Unix tools such as awk, sort, uniq, and grep, which process data efficiently.

```
Program: process_logs.sh
#!/bin/bash
```

```
LOGFILE="access.log"
```

```
awk '
{
    status[$9]++
}
END {
    for (s in status)
        print s, status[s]
}' "$LOGFILE" | sort -n > report.txt
```

```
echo "Log processing completed."
```

README

Problem: One million log entries must be processed quickly using shell scripting.

Solution: awk processes the file in a single pass and stores only required counters. sort is used after aggregation instead of repeatedly sorting the entire log file.

This approach avoids expensive per-line shell processes and is therefore much faster than using commands such as grep inside a shell loop.

For a strict 2-minute requirement, performance should also be verified on the actual server using:

```
time ./process_logs.sh
```

4. Identify, Monitor and Control a Faulty Process

The faulty process must be controlled without rebooting and without disturbing critical services.

```
Program: process_control.sh
#!/bin/bash
```

```
echo "Top CPU-consuming processes:"
ps -eo pid,comm,%cpu --sort=-%cpu | head -6
```

```
read -p "Enter faulty PID: " PID
```

```
echo "Monitoring PID $PID..."
top -b -n 1 -p "$PID"
```

```
echo "Reducing CPU priority..."
sudo renice +10 -p "$PID"
```

```
echo "Process controlled successfully."
README
```

Problem: A faulty background process is consuming excessive CPU resources.

Solution: ps identifies the CPU-heavy process, top monitors it, and renice lowers its CPU scheduling priority. This allows critical services to receive CPU time without restarting the system.

If the process becomes completely unresponsive, an administrator can terminate it selectively:

```
sudo kill -TERM <PID>
```

Only use kill -9 as a last resort.

5. RAM Allocation for 5 Virtual Machines

Available RAM = 16 GB

Required:

```
Host OS = 4 GB
5 VMs × 2 GB = 10 GB
Total minimum = 14 GB
Remaining = 2 GB
```

A practical allocation is:

Resource	RAM
Host OS	4 GB
VM1	2 GB
VM2	2 GB
VM3	2 GB
VM4	2 GB
VM5	2 GB
Reserved/Buffer	2 GB

```
Total 16 GB
Program: vm_allocate.sh
#!/bin/bash
```

```
HOST_RAM=4
VM_RAM=2
VMS=5
TOTAL_RAM=16
```

```
REQUIRED=$((HOST_RAM + VMS * VM_RAM))
REMAINING=$((TOTAL_RAM - REQUIRED))
```

```
echo "Total RAM      : ${TOTAL_RAM} GB"
echo "Host RAM       : ${HOST_RAM} GB"
echo "VMs             : ${VMS}"
echo "RAM per VM       : ${VM_RAM} GB"
echo "Required RAM     : ${REQUIRED} GB"
echo "Reserved RAM     : ${REMAINING} GB"
```

```
if [ "$REMAINING" -ge 0 ]; then
    echo "Allocation is feasible."
else
```

```
    echo "Insufficient RAM."  
fi
```

README

Problem: Five VMs must run on a server with only 16 GB RAM.

Solution: Allocate the minimum 2 GB per VM and reserve 4 GB for the host. This consumes 14 GB, leaving 2 GB as a safety buffer. The hypervisor should use memory overcommit/ballooning only when necessary, while avoiding excessive swapping.